

.NET GC Internals (Compact)

How the .NET / CoreCLR garbage collector works: heap layout, allocation, generations, mark/compact/sweep, cards, LOH/POH, finalization, and Server/Background GC — with diagrams and the facts that matter in practice.

1. What the GC really does

1.1 Why managed memory exists

In native code (C/C++), every heap allocation is the programmer's responsibility:

```
Widget* w = new Widget();  
Use(w);  
delete w;    // you decide when
```

That model fails at scale:

Failure	Consequence
Forgot to free	Leak → growing working set → OOM
Freed too early	Use-after-free → crash / exploit
Freed twice	Heap corruption
Unclear ownership	Lifetime bugs across APIs/threads

Managed runtimes invert the contract: **the language allocates; the runtime reclaims**. You reason about references and meaning, not about who owns a raw pointer.

Managed memory is not “slow memory.” Allocation is often cheaper than `malloc` (bump pointer). Reclaim happens in batches and can introduce pauses or concurrent GC CPU work.

1.2 The question GC answers

Once you forbid explicit free, something must answer:

Which objects are still reachable from the running program?

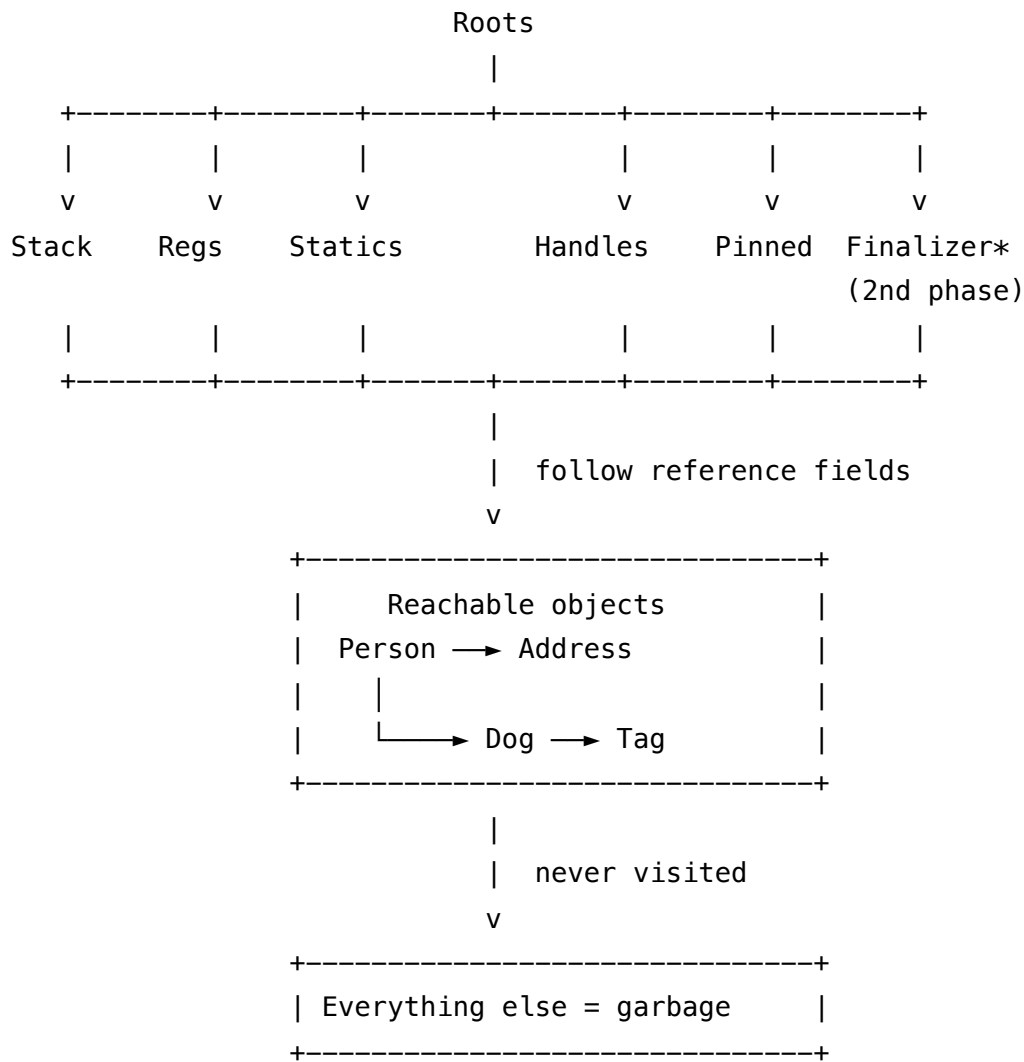
That is the GC's job. Generations, card tables, LOH, background collection — all are optimizations on top of that question.

1.3 The fundamental inversion

The GC does not find objects to delete. It finds objects that are reachable. Everything else becomes garbage.

`alive` = transitive closure of roots under reference edges

`garbage` = `heap` - `alive`



*Finalization keeps otherwise-dead finalizable objects alive long enough to run `Finalize` — it is not a permanent root for every finalizable instance.

If a forgotten static or event handler still holds a reference, the object is **alive**, no matter how “unused” it feels.

1.4 Memory management families

Manual

Allocator knows free blocks; does **not** know whether any pointer still points at them.

Reference counting

Assign: `target->refCount++`

Clear: `target->refCount--; if (0) free(target)`

Immediate reclaim, but cycles ($A \rightarrow B \rightarrow A$) never hit zero without extra logic, and every pointer write costs atomics.

Tracing (what .NET uses)

Start from roots, mark reachable objects, reclaim the rest. Cycles are free: an unrooted cycle is simply never marked.

Generational

Most objects die young → collect a small nursery often; collect old space rarely.

.NET GC summary: precise tracing, generational, mostly compacting on SOH, sweeping LOH by default, workstation/server modes, optional background Gen2.

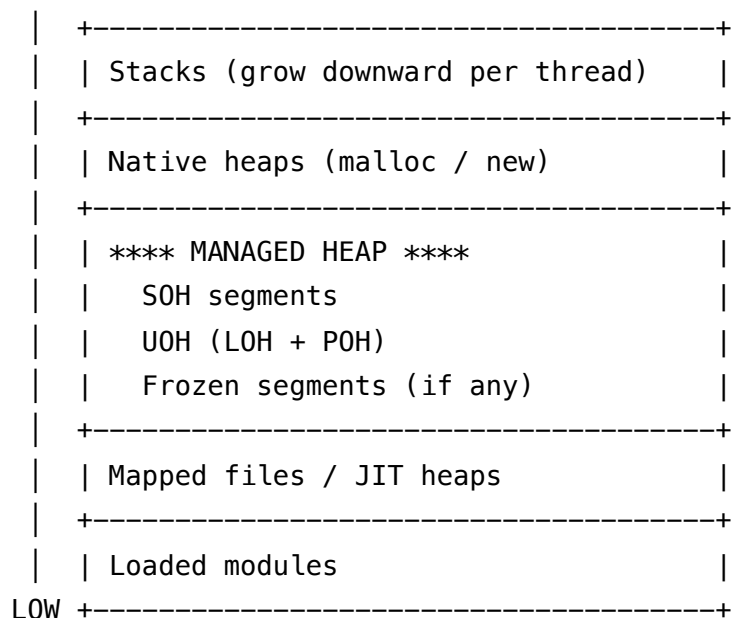
2. Managed heap architecture

2.1 Virtual memory

A process sees **virtual address space**. The OS maps VA pages to physical RAM or pagefile on demand.

Process virtual address space (conceptual)

HIGH addresses



Concept	Meaning
Reserve	Claim VA range; no physical pages yet
Commit	Back pages so they can be touched; counts toward commit charge
Decommit / release	Return unused pages or reservations after GC when policy allows

Why it matters: the CLR can reserve large contiguous ranges and commit as needed; after GC it may decommit tails to shrink working set while keeping VA reserved for growth.

Platform	Typical user VA	GC implication
32-bit	~2 GB (up to ~4 GB with LAA = Large Address Aware)	Fragmentation / LOH holes are deadly
64-bit	Vast	Commit/RAM is the limit, not VA

2.2 Segments and the ephemeral segment

The managed heap is a set of **segments**, not one flat arena.

What “ephemeral segment” means

Ephemeral = short-lived. In the CLR, **Gen0** and **Gen1** are the ephemeral generations — objects that are expected to die soon (or survive only briefly before becoming “old”).

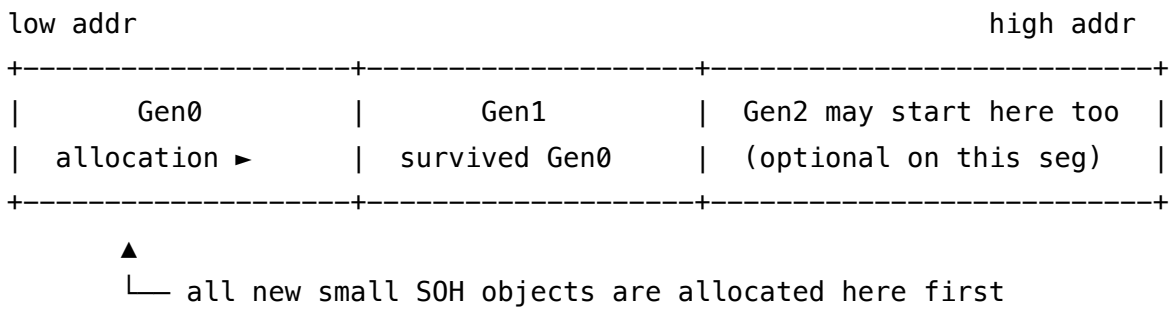
The **ephemeral segment** is the single contiguous SOH memory range where those young generations live:

Lives on ephemeral segment	Does not (normally)
Gen0 — brand-new small objects	Extra Gen2-only segments
Gen1 — survivors of Gen0	LOH / POH (separate UOH segments)
Sometimes the start of Gen2 on the same segment	Frozen segments

Why one dedicated segment?

- Almost all new (small) traffic hits Gen0 here — bump allocation stays simple and local.
- An **ephemeral GC** (Gen0 or Gen1) only needs to collect this young area (plus dirty cards from old space), not the whole Gen2 heap.
- There is **exactly one ephemeral segment per SOH heap** (Workstation: one heap → one ephemeral segment; Server: one per gc_heap).

Ephemeral segment (young objects)



After collections, survivors are promoted (Gen0→Gen1→Gen2). Gen2 may spill onto **additional segments** that contain only old objects. Empty SOH segments can sometimes be **reused** as a new ephemeral segment instead of asking the OS for more VA. Segments generally grow low address → high address. Initial SOH and LOH segments are reserved when the runtime starts.

2.3 Generations — public API vs internals

What C# exposes: three generations

```
GC.MaxGeneration == 2           // 0, 1, 2
GC.GetGeneration(obj)           // returns 0, 1, or 2
GC.Collect(2)                    // full GC (SOH Gen2 + UOH)
```

Public gen	Contents
0	Youngest SOH — new small allocations
1	Survivors of Gen0
2	Long-lived SOH; API also reports 2 for LOH/POH objects

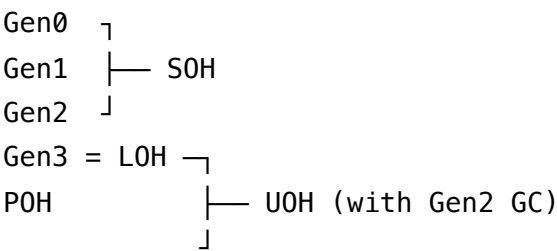
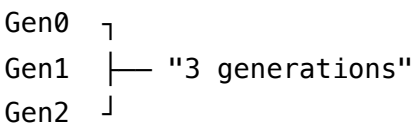
What CoreCLR tracks internally

LOH is **internal generation 3**. POH is a separate UOH heap. Both are condemned with Gen2 — never with Gen0-only collections.

Internal	Heap	GC.GetGeneration
0	SOH Gen0	0
1	SOH Gen1	1
2	SOH Gen2	2
3	LOH	2 (API hides 3)
UOH	POH	2

Public / interview model

Collector model



Collecting generation *N* always collects 0..*N*. Gen2 also collects UOH.

2.4 Heap roles

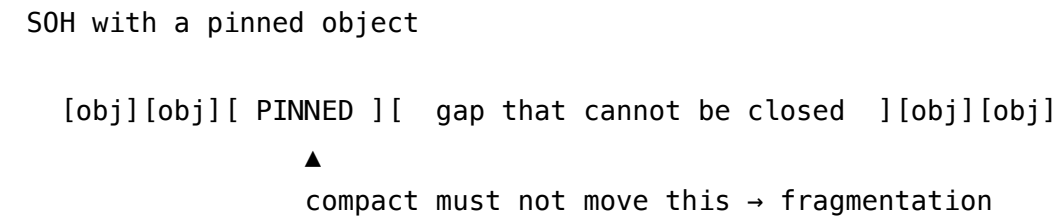
-----+	-----+	-----+
SOH	UOH (with Gen2)	Frozen
Gen0 \ Gen1	LOH (large) \ POH	segs
Gen2		
-----+	-----+	-----+

Heap	Role
SOH	Small objects; allocation contexts; often compacted
LOH	≥ ~85KB; free lists; collected with Gen2; sweep by default
POH	Objects pinned at allocation ; collected with Gen2
FOH/frozen	Immortal, immovable registered data — never collected

UOH = LOH + POH — user code can allocate here directly; distinct from SOH Gen2 (which only receives *promotions*).

POH — why it exists

Problem: pinning an object on the **SOH** (`fixed` , `GCHandleType.Pinned` , interop marshalling) makes that object **immovable**. During compacting GC the collector must leave it in place as a **plug**, which forces free **gaps** around it:



A few short-lived `fixed` blocks are fine. Many **long-lived** pins (high-throughput interop, networking buffers, some serializers) permanently wreck SOH compaction quality — Gen0/Gen1 pauses get worse and the heap fragments.

Solution (.NET 5+): Pinned Object Heap

Allocate the buffer **already pinned** into a separate heap so pins do not sit inside the compacting SOH:

```
byte[] buffer = GC.AllocateArray<byte>(4096, pinned: true); // lands on POH
```

Behavior	Detail
When used	Pin requested at allocation time
Not used for	Pinning an <i>existing</i> SOH object — <code>fixed</code> / <code>GCHandle.Pinned</code> still pin in place on SOH
Collection	With Gen2 (part of UOH), like LOH
Compaction	POH is not compacted like SOH — pins are expected here
Allocation	Free-list / UOH style — not the SOH bump-pointer path
API gen	<code>GC.GetGeneration</code> reports 2

With POH

```
SOH: [obj][obj][obj][obj]...    ← still compactable
POH: [pinned][pinned][pinned]... ← pins isolated
```

Takeaway: use POH for buffers you know must stay pinned for a long time; keep `fixed` scopes short when you only need a temporary pin on SOH data.

FOH — why it exists

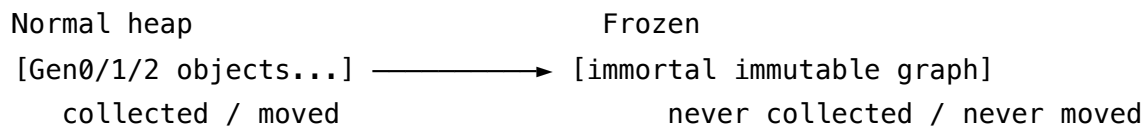
Problem: some data is **truly immortal and immutable** for the process lifetime (or a registered frozen segment): precomputed tables, const-like graphs, NativeAOT / ready-to-run frozen blobs, large read-only datasets. Putting that on a normal heap means:

- the GC must still *consider* it during collections
- it may be relocated during compact (wasted work)
- it adds to Gen2 size and mark cost even though it never dies

Solution: Frozen Object Heap / frozen segments

Register a range of already-initialized objects as **frozen**:

Behavior	Detail
Lifetime	Never collected — treated as immortal
Relocation	Never moved — immovable by design
Allocation	Not a general new target for app DTOs
References	Normal heap objects may point <i>into</i> frozen memory
Barriers / GC	Runtime treats frozen ranges as non-evacutable; no need to reclaim them



Important: FOH is **not** “any record or immutable class automatically goes here.” Ordinary immutable objects still live on SOH/LOH. Frozen segments are for runtime-managed or explicitly registered ready-only data (including scenarios used heavily around NativeAOT and modern .NET frozen-object support).

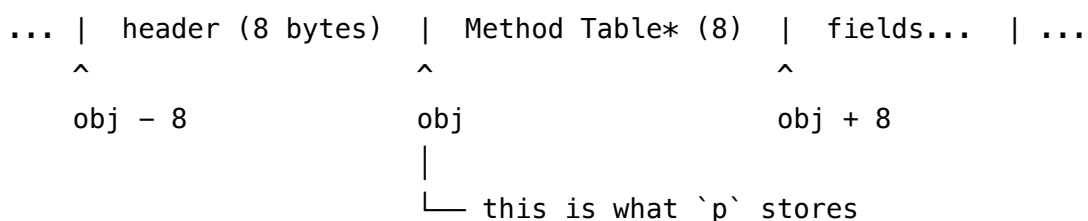
Takeaway: POH fixes **pin fragmentation** on the compacting heap; FOH fixes **pointless GC work** on data that will never die or move.

3. Object memory layout

When you hold a reference in C# (`Person p = ...`), that reference is a pointer to a place in the heap. Call that address **obj** .

On **x64**, the bytes around `obj` look like this:

Memory addresses increase →



Offset	What is there	Size on x64
<code>obj - 8</code>	Object header (sync block / hash / lock bits)	8 bytes <i>before</i> the reference
<code>obj /</code> <code>obj + 0</code>	Pointer to the type's Method Table	8 bytes — where the reference points
<code>obj + 8</code>	Start of instance fields (Name , Age , ...)	depends on the type

So `obj-8` , `obj+0` , `obj+8` are not special CLR names — they are just **byte offsets from the address stored in your reference**:

- subtract 8 → header
- add 0 → Method Table pointer (the reference itself)
- add 8 → first field

On **x86** the same layout uses **4-byte** pointers, so the header is at `obj - 4` and fields start at `obj + 4` .

Same idea, drawn as a box:

ObjHeader	Method Table*	Instance fields...
(before the ref)	← reference points	Name, Age, ...

Array sketch (fields include length, then elements):

Header	MT*	length	e0	e1	e2	...
--------	-----	--------	----	----	----	-----

▲
reference points here

Method Table

From the MT the runtime gets type identity, size/component size, vtable, flags, and the **GC descriptor** (which fields/elements are references).

NativeAOT's EEType is related but separate — CoreCLR SOS/ gc.cpp talk about MethodTables.

GC descriptor (conceptually)

```
class Person
{
    string Name;      // reference
    int Age;          // value
    Address Address;  // reference
}
```

During mark, for each live `Person`, the collector walks only reference slots from the descriptor — never treating `Age` as a pointer.

Embedded structs with refs are flattened into the containing object's descriptor. `object[]` is “all elements are refs.”

4. Allocation internals

```
var person = new Person();
```

Conceptual pipeline:

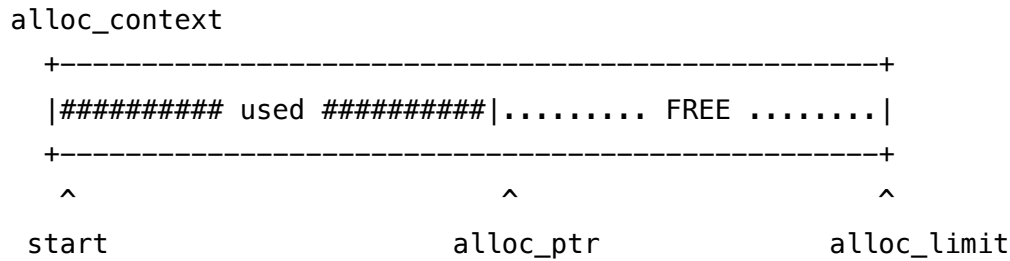
1. Type/MT already loaded
2. Compute aligned total size
3. $\text{size} \geq \text{LOH threshold?} \rightarrow \text{LOH} : \text{SOH Gen0}$
4. Bump from thread allocation context (SOH)
5. Memory already zeroed
6. Write Method Table*
7. Register finalizer if needed
8. Call `.ctor`
9. Return reference

Notes:

- LOH threshold compares **total object size** (including overhead), not only `byte[]` length.

- Threshold is configurable (`GCL0HThreshold`).
- LOH skips allocation contexts entirely.

4.1 Bump pointer



Step by step:

(1) quantum ready, zeroed

```
|_____|
```

(2) new A

```
|AAAA|_____|
```

(3) new B

```
|AAAA|BBBBBB|_____|
```

(4) next does not fit → leftover FREE object, new quantum

```
|AAAA|BBBBBB|FREE| >>> new quantum >>>
```

Allocate(size):

```

size = Align(size)
if alloc_ptr + size > alloc_limit:
    return slow_path    // new quantum or trigger GC
result = alloc_ptr
alloc_ptr += size
return result

```

Why fast: no free-list search, thread-private, JIT-inlined, contiguous young objects.

4.2 Alignment and minimum size

Arch	Typical align
x86	4-byte
x64	8-byte

Empty classes still pay header + MT (+ min size). Free-object representation requires gaps $\geq$ `min_obj_size` (commonly ~ 3 pointers).

5. When does GC start?

Not only “RAM is full.”

Allocation budgets

Each generation has a budget — how much allocation/survival is tolerated before condemning that generation. Gen0 budgets are **dynamic**.

Gen0 budget (illustrative)

`~256 KB → a few MB → tens of MB (workload / survival / DATAS)`

High death rate $\rightarrow$ larger budget OK (fewer GCs). High survival $\rightarrow$ collect sooner before junk promotes into Gen1/Gen2.

Triggers

Trigger	What happens
Gen0 budget exceeded	Most common ephemeral GC
Gen1/Gen2 thresholds	Deeper collections
<code>GC.Collect</code>	Induced

Trigger	What happens
Large alloc cannot fit	May trigger GC before OOM
OS / container pressure	More aggressive policy
Fragmentation (esp. LOH)	May compact or deepen collection
GC.AddMemoryPressure	Native memory associated with managed wrappers

GC may run **while free memory still exists** because budgets, survival stats, or pressure say waiting is worse.

Latency modes

```
GCSettings.LatencyMode = GCLatencyMode.Interactive;           // default feel
GCSettings.LatencyMode = GCLatencyMode.LowLatency;            // postpone Gen2 briefly
GCSettings.LatencyMode = GCLatencyMode.SustainedLowLatency;
GCSettings.LatencyMode = GCLatencyMode.Batch;                 // throughput
```

These bias *when/which* collections happen — they do not disable GC.

DATAS (modern Server GC): adapts heap size / Gen0 budget toward live data size — important in containers. Details in the full article.

6. Roots and tracing

.NET is **precise**: at safe points the **EE (Execution Engine)** — the CLR component that runs managed code, knows JIT GC info, and coordinates with the GC — reports exact ref locations (not “every bit pattern that looks like a pointer”).

Root enumeration

Root	Who knows locations	Notes
Stack slots / spills	JIT GC info	Dead locals should not appear

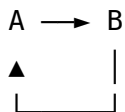
Root	Who knows locations	Notes
CPU registers	GC info + thread context	
Statics	Loader / type system	Collectible ALCs can drop them
GCHandle strong/pinned	Handle table	Explicit roots
Weak / WeakReference	Handle table	Not strong roots
Finalization	Queue after normal mark	2nd-phase keep-alive
Dirty cards	Card table	Old→young during ephemeral GC
Dependent handles	Special scan order	e.g. ConditionalWeakTable

Graphs

Root → Order → Lines[] → Product

Unreachable: Orphan → Ghost ← garbage

Cycles



Mark bit stops re-entry. Unrooted cycles die. This is why tracing beats naive reference counting for graphs with cycles.

Interior pointers

fixed , byrefs, and pinned addresses may point **inside** an object. Marking maps them back to the object base before setting the mark bit.

7. Safe points and suspension

Safe points

A location where:

1. Live refs are accurately known
2. The thread may stop for GC

JIT inserts polls / uses call sites. Infinite loops without calls still need GC polls.

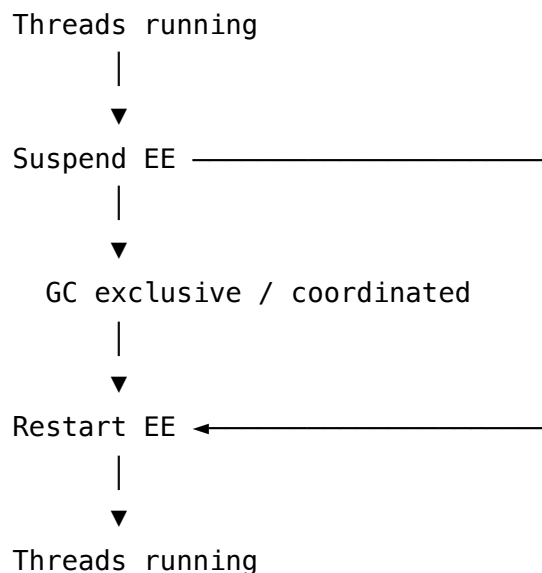
Cooperative vs preemptive

Mode	Meaning
Cooperative	Managed code; must reach safe point
Preemptive	Typically native/blocked; GC need not wait on that thread's poll

P/Invoke usually goes preemptive so GC can proceed during long native calls — incorrect interop that hides managed refs is dangerous.

Suspend EE / Restart EE

EE = Execution Engine. Suspending it means: stop managed threads at safe points so the GC can scan roots and move/reclaim objects safely.



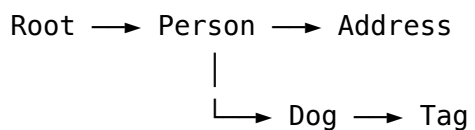
Background GC reduces how often Gen2 needs a long blocking suspend, but ephemeral GCs and some BGC phases still suspend briefly.

8. Mark, plan, compact, sweep

8.1 Mark phase

Goal: compute reachable set.

Object graph



Unreachable: Orphan

Mark stack (explicit — **not** native recursion):

```
push Person
pop Person → mark → push Address, Dog
pop Dog → mark → push Tag
...
empty → done
```

Mark array: P A D T 0 → 1 1 1 1 0

Server GC: parallel mark + work stealing across heaps.

8.2 Plan chooses compact vs sweep

Wrong: “SOH always compacts; LOH always sweeps.”

Right: after mark, **plan** decides (`decide_on_compacting` in CoreCLR).

```
mark_phase()
plan_phase()    // simulate compact: plugs from pins, new addresses, reclaimable gaps
|
├ compact → relocate_phase() → compact_phase()
└ sweep   → make_free_lists()
```

How the decision is made

Plan does **not** flip a coin. It **simulates** compaction (accounting for pinned plugs), measures how much free space that would reclaim, then asks: is compact worth it?

Forced compact (skip the heuristic) when any of these apply:

Force	Example
API	<code>GC.Collect(..., compacting: true)</code> / aggressive collect
Config	<code>DOTNET_GCForceCompact</code> / stress heap
Near OOM	last GC before OOM
Space pressure	low ephemeral space; cannot leave free gaps
High / very high memory load	machine RAM under pressure + enough reclaimable Gen2

Otherwise — fragmentation heuristic (percentage **and** absolute size):

fragmentation_burden = planned_free_after_compact / generation_size

compact if:

```
fragmentation >= fragmentation_limit    // absolute bytes
AND burden    >= fragmentation_burden_limit // ratio
```

Default limits from CoreCLR static_data (balanced latency; can vary by latency level):

Gen	Absolute floor	Burden (≈ %)
Gen0	~40 KB	50%
Gen1	~80 KB	50%
Gen2	~200 KB	25%

So for Gen2: “about **25%** of the generation would become free **and** that free space is at least ~200 KB” → compact. Small holes that are a tiny % of Gen2 → often **sweep**.

Other notes:

Situation	Typical
Blocking SOH	Compact or sweep after plan (above)
LOH / POH	Sweep by default (LOH compact only if mode / hard limit asks)
Background Gen2	Concurrent mark + sweep — never compact (never runs this decision)
Heavy SOH pins	Plugs shrink compact ROI → more likely to sweep or leave gaps

8.3 Compact path

PLAN new addresses:

```
old: | A | gap | B | gap | C |
new: | A | B | C |.....|
```

RELOCATE references (roots + heap fields)

COMPACT copy bytes (gcmemcopy)

With pins:

```
| A | gap | [PINNED] | gap | B | C |  
      ▲ plug cannot move
```

8.4 What “sweep” means

After **mark**, the GC knows which objects are alive (marked) and which are dead (unmarked). It then has two ways to reclaim the dead space:

Approach	What it does	Objects move?
Sweep	Walk the heap, turn unmarked gaps into free blocks , put them on a free list for later reuse	No — live objects stay at the same addresses
Compact	Slide live objects together, then fix all references	Yes — addresses change

Sweep = “leave survivors where they are; record the holes as free memory.”

After mark:

```
| A (live) | B (dead) | C (live) | D (dead) | E (live) |  
| 1       | 0       | 1       | 0       | 1       |
```

After sweep (objects did not move):

```
| A (live) | FREE  | C (live) | FREE  | E (live) |  
      ▲           ▲  
      dead space becomes free-list entries  
      (adjacent dead runs can be merged into one FREE)
```

Free list (conceptual): $\text{FREE}(B) \rightarrow \text{FREE}(D) \rightarrow \dots$

Next allocation can reuse a hole that is big enough.

Why use sweep instead of compact?

- **Cheaper** when moving would be expensive (especially **LOH** — huge arrays).
- **Background Gen2** sweeps concurrently and does **not** compact.
- Plan may choose sweep on SOH too if compact would not pay off (e.g. heavy pinning).

Downside of sweep: **external fragmentation** — many holes, but no single hole large enough.

```
| A | FREE 40K | C | FREE 80K | E |  
Need 100K contiguous → fail even if total free > 100K
```

Compact fixes that by sliding live objects together:

```
Sweep result:      A gap C gap E  
After compact:    A C E FREE.....
```

8.5 Fragmentation types

Fragmentation means “bytes exist but are hard to use.” .NET cares about two kinds — and a third **pin-shaped** form that looks like external.

Internal fragmentation

Waste **inside** an already-allocated object slot. The object owns those bytes; they cannot satisfy another new .

```
Requested payload: 18 bytes  
Aligned allocation: 24 bytes (pointer-size / layout rules)
```

```
|##### payload #####| pad |  
|<----- 24 bytes ----->|  
    ▲ owned by this object, not on any free list
```

Where it shows up in .NET:

Source	What happens
Alignment	Objects are pointer-aligned; odd sizes get pad
Object header	Method table + sync-block index cost even for tiny types
Array / struct layout	Padding between fields or at the end of a type
LOH free-list split leftovers	Tiny leftover free objects after a hole is partially reused

Internal fragmentation is usually a **small, steady tax**. Compaction does **not** fix it — the object still needs its aligned size.

External fragmentation

Free bytes exist **between** live objects, but not as one contiguous run large enough for the next allocation.

After sweep (or after pins leave gaps):

```
+-----+-----+-----+-----+-----+-----+
|  A  | FREE  |  C  |  FREE  |  E  | FREE  |
|live| 40KB |live|  80KB |live| 30KB |
+-----+-----+-----+-----+-----+-----+
```

Total free $\approx$ 150KB

Largest hole = 80KB

Need 100KB contiguous $\rightarrow$ fail / Gen2 / grow heap
(even though “enough” free bytes exist)

Why it hurts:

- Allocations trigger GC or grow the heap while free lists look rich.
- Free-list allocators (especially **LOH**) must find a hole that **fits**; tiny holes are often useless.
- On **32-bit**, address-space holes matter; on 64-bit the pain is usually **commit / working set**, not VA.

Compact is the main cure: slide live objects together $\rightarrow$ one free region at the end (modulo plugs).

```
Fragmented:  [A] [gap] [B] [gap] [C] [gap]
Compacted:   [A] [B] [C] [          free at end          ]
```

Pin / plug fragmentation (SOH)

A pinned object cannot move. During compact it stays as a **plug**; free **gaps** appear around it even if the collector wanted a clean slide.

```
Wanted:      [A] [B] [C] [D] [ FREE..... ]
With pin:    [A] [ FREE ] [ PINNED ] [ FREE ] [B] [C]
               ▲ plug
```

Those gaps are **external** fragmentation caused by immobility. Heavy long-lived `fixed / GCHandle.Pinned` on SOH wrecks compact quality. That is why **POH** exists: put pins out of the compacting nursery.

Sweep vs compact vs BGC (who leaves holes?)

Path	External fragmentation
Compact	Clears most holes (except around plugs)
Sweep	Leaves holes; builds free lists
Background Gen2	Sweep only → Gen2/LOH can stay fragmented for a long time
LOH default	Sweep → classic “lots of free, no hole big enough”

Production pattern: Gen2 / LOH size huge, promoted bytes small, many BGCs → often **fragmentation**, not a managed leak. A blocking compacting full GC (and LOH compaction if needed) is the check.

How to think about “frag %” in tools

Traces often show something like:

$$\text{frag \%} \approx \text{free / fragmentation bytes in gen} \div \text{generation size}$$

High Gen0 frag % after pins is common (and sometimes intentional — free space is immediately reusable for new allocs). High **Gen2 / LOH** frag % with a growing working set is the one that usually needs compacting attention.

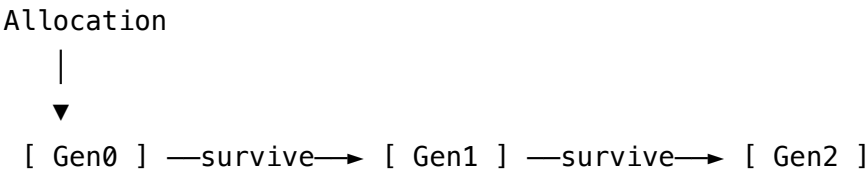
Quick map

Kind	Where waste sits	Fixed by compact?
Internal	Inside an object’s slot	No
External	Holes between objects	Yes (usually)
Pin / plug	Forced gaps around immovable objects	Only partly (plug stays)

9. Generational GC

Why

Most objects die young → collect nursery often and cheaply; collect Gen2 rarely.

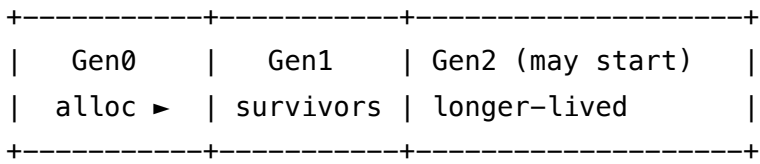


Public generations (the C# model)

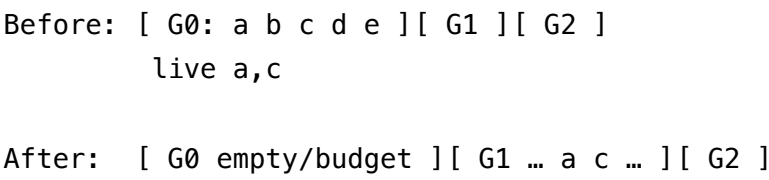
Gen	Role
0	Nursery — all new small SOH objects
1	Holding area — Gen0 survivors
2	Long-lived small objects

Sizes are **dynamic**, not fixed constants. Gen0 is often hundreds of KB to several MB+ depending on workload and policy.

Ephemeral segment picture



After Gen0 GC:



Nesting

Gen0 → Gen0 only

Gen1 → Gen0 + Gen1

Gen2 → Gen0 + Gen1 + Gen2 + LOH + POH

Promotion default: survive $N \rightarrow N+1$. Gen2 survivors stay Gen2. LOH survivors stay on LOH.

Demotion/non-promotion can occur as policy exceptions.

Address space

- **32-bit:** Gen2 + LOH growth/fragmentation historically common killers.
- **64-bit:** Gen2 can be multi-GB; limited by commit/RAM, not a fixed “Gen2 max.”

Remember: **3 public generations**. LOH-as-gen3 is internal numbering only.

10. Card table and write barrier

Problem

Gen2 Person $\longrightarrow$ Gen0 Dog

Ephemeral GC must see this edge without scanning all of Gen2.

Write barrier

```
person.Child = dog;
```

JIT emits a barrier (helpers like `JIT_WriteBarrier` variants).

Generational rule: if the **stored value** is in the **ephemeral** range, dirty the **card** for the **slot address** (where the reference is written).

```
WriteBarrier(slot, value): // educational
    *slot = value
    if value points into ephemeral:
        DirtyCard(CardIndex(addressof(slot)))
```

Also interacts with background/concurrent marking so new edges are not missed.

Card table

Classic coverage ~**2 KB** heap per card.

```
Heap:  |--card0--|--card1--|--card2 (field)--|--card3--|
Cards: | clean   | clean   | DIRTY               | clean   |
```

Brick tables help find object starts when scanning a dirty card.

Ephemeral GC use

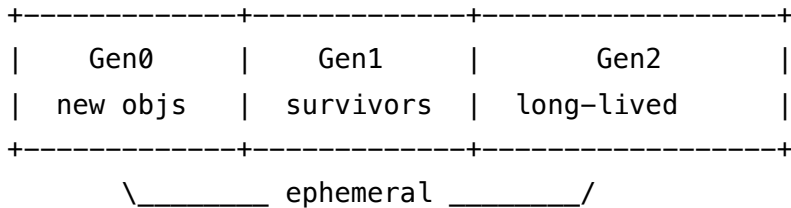
1. Mark from EE roots
2. Scan dirty cards in old/UOH for refs into ephemeral
3. Mark those objects
4. Reset cards as appropriate

```
Full Gen2 scan:  0(|Gen2|)          ← unaffordable every Gen0
Dirty card scan: 0(dirty×card)      ← tracks mutation rate
```

False positives exist (dirty but no young refs) — cheap barriers, slightly extra scan.

11. SOH, LOH, POH, FOH

SOH



- Bump allocation via contexts
- Plan may compact or sweep
- Cards cover old→young

LOH

Default threshold **85,000** bytes total size (configurable).

```
new byte[1_000]; // SOH → GetGeneration 0
new byte[85_000]; // LOH → GetGeneration 2 (internal gen3)
```

Topic	Behavior
Why separate	Moving huge objects is expensive
Allocation	Free list / grow — no context
Collection	Only with Gen2
Default reclaim	Sweep → free lists
Compaction	Off by default; CompactOnce / heuristics
Fragmentation	Classic production issue

LOH holes

| 2MB | FREE 200K | 512K | FREE 700K | 1MB | FREE 100K |

Free list conceptually: 700K → 200K → 100K

Allocate 600K → may split 700K hole

Allocate 850K → may grow even if total free > 850K

```
GCSettings.LargeObjectHeapCompactionMode =  
    GCLargeObjectHeapCompactionMode.CompactOnce;  
GC.Collect(2, GCCollectionMode.Forced, blocking: true, compacting: true);
```

POH (.NET 5+)

Without POH (pin SOH object):

SOH: [obj][PINNED][obj][gap forced][obj]

With POH (pin-at-alloc):

SOH: [obj][obj][obj]... compactable

POH: [pinned][pinned]...

```
GC.AllocateArray<byte>(4096, pinned: true); // POH
```

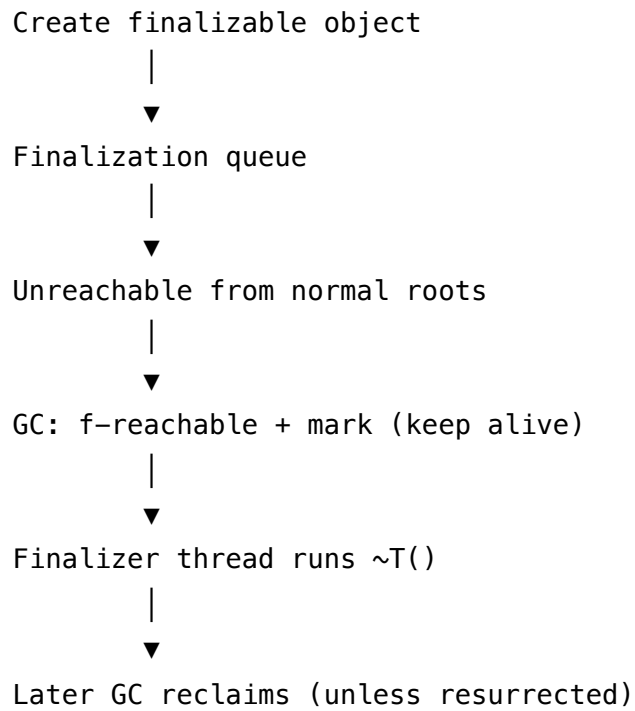
fixed / GCHandleType.Pinned on an existing SOH object pins **in place** — does not migrate to POH.

FOH / frozen segments

Immortal, immovable registered ranges. Not a dumping ground for ordinary immutable DTOs. Useful for large ready-only graphs.

12. Finalization

Not deterministic destructors.



Consequences:

- ≥ 1 extra GC after finalization before memory returns
- Finalizer thread: a blocking finalizer stalls the process's finalization
- Prefer `IDisposable` / `using` / `SafeHandle`
- `GC.SuppressFinalize(this)` in `Dispose`

Resurrection

```
~MyType()  
{  
    SomeStatic.Instance = this; // object lives again  
}
```

Teaching anti-pattern. Memory is not freed; finalizer will not run again unless `GC.ReRegisterForFinalize`.

13. Workstation, Server, Background GC

Workstation vs Server

	Workstation	Server
Heaps	Typically 1	Many gc_heap s
Goal	Latency / desktop	Throughput
GC work	Often on triggering thread (non-BGC)	Dedicated GC threads
Memory	Leaner baseline	Higher (esp. pre-DATAS)

```
<ServerGarbageCollection>true</ServerGarbageCollection>
```

```
{
  "runtimeOptions": {
    "configProperties": {
      "System.GC.Server": true
    }
  }
}
```

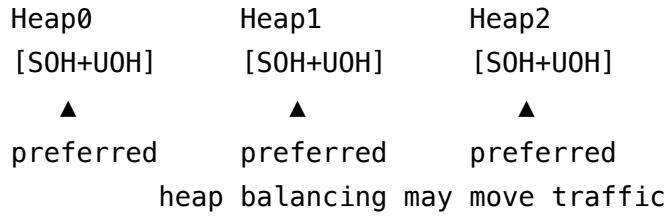
```
DOTNET_gcServer=1
```

```
# compare:
```

```
DOTNET_gcServer=0 dotnet run --project 12.ServerGcConfig
```

ASP.NET hosting often enables Server GC — check before assuming Workstation.

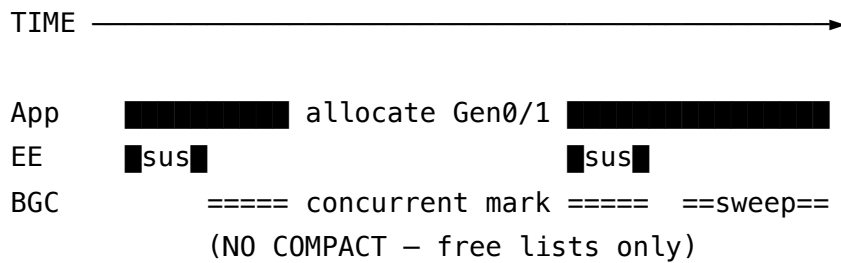
Server GC



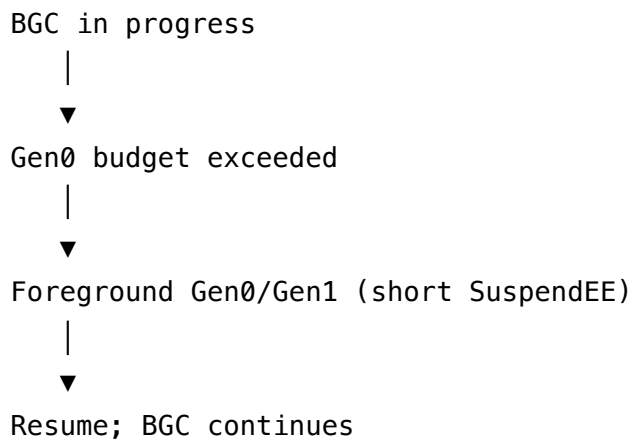
GC threads mark/plan/compact in parallel after SuspendEE

Background GC

Term	Meaning
Blocking GC	Substantial SuspendEE; may compact
Background GC	Gen2 concurrent mark + sweep; no compact
Foreground GC	Ephemeral Gen0/1 during an in-flight BGC



If Gen0 fills mid-BGC:



Production pattern: Gen2/LOH size huge, promoted bytes small, many background GCs → often **fragmentation**, not a leak. Confirm with blocking compacting full GC + traces.

14. Diagnostics

Tool	Use
dotnet-counters	Live alloc rate, heap size, gen counts, time in GC
dotnet-trace / PerfView	Per-GC reason, gen, pause, BGC vs blocking, survival
dotnet-gcdump	Managed heap snapshot
SOS	!eeheap -gc , !dumpheap -stat , !gcroot , !gchandles

```
dotnet-counters monitor System.Runtime -p <pid>
```

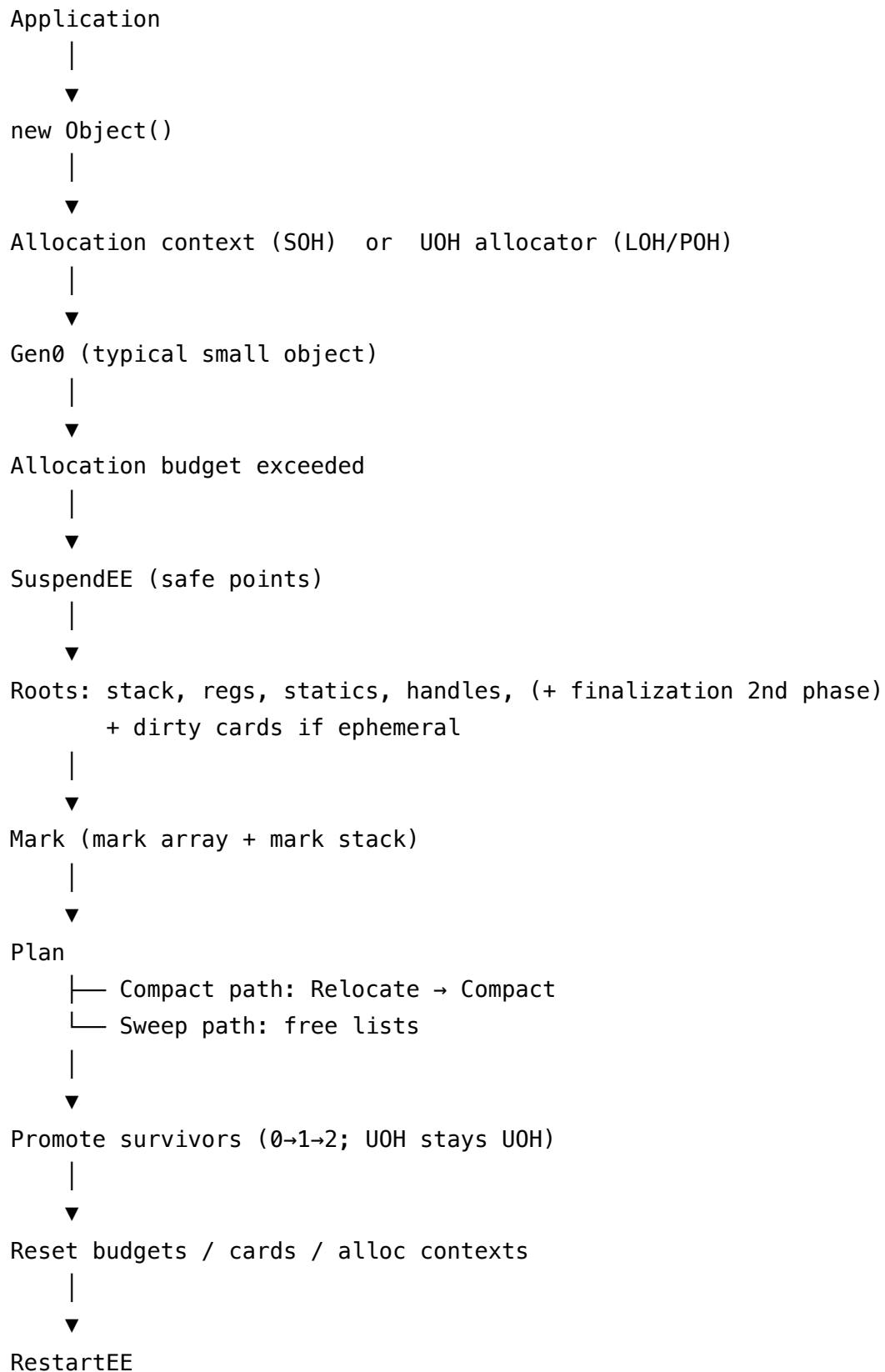
Watch: Allocation Rate, GC Heap Size, Gen 0/1/2 Collections, Time in GC, LOH size.

Per GC in a trace, ask:

1. Reason?
2. Generation condemned? (2 includes UOH)
3. Background or blocking?
4. Compaction?
5. Survival / promotion volume?

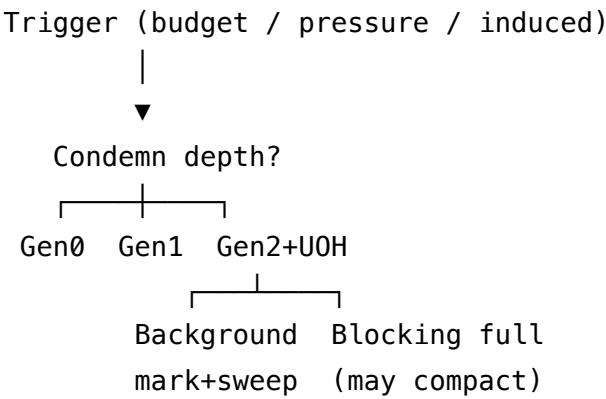
High Gen0 rate with tiny pauses can be healthy churn. Frequent Gen2 with rising heap and low survival → investigate caches, leaks, LOH holes, pins.

15. End-to-end flow



Gen2 may instead run as **Background GC** (concurrent mark/sweep, no compact), with foreground ephemeral GCs allowed concurrently.

Expanded decision:



16. Common misconceptions

Myth	Reality
GC frees as soon as I'm "done"	Only at collection via reachability
GC.Collect is good architecture	Usually harms budgets/promotions
Generation = age in seconds	Survival across collections
Gen2 = permanent	Collected on Gen2 GCs
4 public generations (LOH gen3)	API has 3 (0–2); LOH internal gen3
Ref count determines generation	Location + promotion does
LOH never collected	With Gen2; often not <i>compacted</i>
BGC keeps Gen2 tidy	BGC does not compact
All pins go to POH	Only pin-at-alloc
Finalizers are cheap dtors	Extra lifetime + finalizer thread
list.Clear() frees elements	Drops refs only

Myth	Reality
Server GC always better	Throughput vs memory/latency tradeoff
TryGetTarget before collect in demos	Can leave a strong stack root

Appendix A — Worked scenarios

A.1 Promotion Gen0 → Gen1 → Gen2

```
object x = new byte[1024];
Console.WriteLine(GC.GetGeneration(x)); // 0

GC.Collect(0, GCCollectionMode.Forced, blocking: true);
Console.WriteLine(GC.GetGeneration(x)); // 1 (if still rooted)

GC.Collect(1, GCCollectionMode.Forced, blocking: true);
Console.WriteLine(GC.GetGeneration(x)); // 2

GC.KeepAlive(x);
```

Internals: bump alloc into Gen0 → keep strong root → Gen0 mark finds x → plan/compact|sweep → promote to Gen1 → same for Gen1→Gen2.

Phases: induced ephemeral then deeper collections: SuspendEE → mark → plan → compact|sweep → promote → RestartEE.

A.2 Unrooted graph dies; rooted cycle lives

```
Rooted:    static → A ↔ B    → both live (mark bit stops cycle walk)
Unrooted:  A ↔ B only      → both die (never reached from roots)
```

Tracing GC does not leak cycles. Weak references alone do not keep targets alive.

A.3 LOH vs Gen0

```
var large = new byte[85_000];  
Console.WriteLine(GC.GetGeneration(large)); // 2  
  
GC.Collect(0, ...); // large still allocated (LOH not condemned)  
// drop root  
GC.Collect(2, ...); // LOH swept – object reclaimable
```

Generation: API reports 2; internal gen3.

Phase: Gen2 mark → plan → typically LOH sweep.

A.4 Pinning

```
// SOH pin – creates plug  
var h = GCHandle.Alloc(buffer, GCHandleType.Pinned);  
  
// POH – pin at allocation  
var poh = GC.AllocateArray<byte>(4096, pinned: true);
```

SOH pins hurt compaction. POH isolates long-lived pinned buffers.

A.5 Finalization vs Dispose

Finalizer path: extra GC + async thread + ordering hazards

Dispose path: deterministic; SuppressFinalize avoids finalizer work

Use `SafeHandle` for OS handles when possible.

Appendix B — Configuration cheat sheet

Goal	Knob
Server GC	System.GC.Server / <ServerGarbageCollection> / DOTNET_gcServer
Background GC	System.GC.Concurrent / <ConcurrentGarbageCollection> (usually on)
LOH compact next full GC	GCSettings.LargeObjectHeapCompactionMode = CompactOnce
LOH threshold	GCLoHThreshold
Container hard limit	GCHeapHardLimit / percent variants
Latency bias	GCSettings.LatencyMode
DATAS on/off	GCDynamicAdaptationMode (see full article)

```
<!-- example csproj -->
<PropertyGroup>
  <ServerGarbageCollection>true</ServerGarbageCollection>
  <ConcurrentGarbageCollection>true</ConcurrentGarbageCollection>
</PropertyGroup>
```

Appendix C — Quick phase cheat sheet

Name	Does	Typical when
Mark	Find reachable objects	Every GC
Plan	Decide compact vs sweep; compute plugs	Blocking GCs
Relocate	Fix references to new addresses	Compacting path
Compact	Move object bytes	Compacting path

Name	Does	Typical when
Sweep	Build free lists from dead space	LOH default; SOH if plan says so; BGC Gen2
Promote	Survivors Gen0→1→2	After ephemeral/full as applicable
BGC mark/sweep	Concurrent Gen2	Most full GCs on modern apps

Ephemeral GC cost drivers

- size of Gen0/Gen1
- number of dirty cards
- survival / promotion volume
- pins in ephemeral space

Full / Gen2 cost drivers

- live old data size
- fragmentation (esp. if must compact)
- LOH size / free-list quality
- BGC vs blocking choice

Mental model

1. **Precise tracing from roots** defines life.
2. **Three public generations**; nursery does most work.
3. **Write barrier + cards** make ephemeral GC scale.
4. **Plan chooses compact vs sweep**; **BGC never compacts Gen2**.
5. **LOH/POH are UOH** — collected with Gen2, not Gen0.
6. Prefer **Dispose / SafeHandle** over finalizers.